

■ LEVEL 17 • GREEDY

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Make a locally safe choice with a proof that it does not hurt the global optimum.
Reference Examples	Activity Selection • Fractional Knapsack • Jump Game • Interval Scheduling • Minimum Platforms
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 17 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Assign Cookies	Greedy + Sorting	#455
2	Jump Game	Greedy Reachability	#55
3	Jump Game II	Greedy Range	#45
4	Best Time to Buy and Sell Stock II	Greedy	#122
5	Partition Labels	Greedy + Last Occurrence	#763
6	Activity Selection	Greedy + Sorting	Absent
7	Jump Game	Greedy Reach	55
8	Jump Game II	Greedy Farthest Boundary	45
9	Gas Station	Greedy Reset	134
10	Assign Cookies	Sort + Greedy	455

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Assign Cookies

Pattern: Greedy + Sorting

LeetCode: #455

■ Input	children greed factors and cookie sizes
■ Output	maximum number of content children
■ Constraints	positive values
■ Main Idea / Pattern	Sort both. Give the smallest cookie that can satisfy the current least-demanding child.
■ Time Complexity	$O(n \log n)$
■ Space Complexity	$O(1)$ excluding sort

■ Optimal C++ Code

```
int findContentChildren(vector<int>& g, vector<int>& s) {  
    sort(g.begin(), g.end());  
    sort(s.begin(), s.end());  
  
    int i = 0;  
    for (int cookie : s) {  
        if (i < g.size() && cookie >= g[i])  
            i++;  
    }  
    return i;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Jump Game

Pattern: Greedy Reachability

LeetCode: #55

■ Input	non-negative jump lengths
■ Output	whether last index is reachable
■ Constraints	$1 \leq n \leq 10^4$
■ Main Idea / Pattern	Track the farthest reachable index. If current i is beyond it, progress is impossible.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
bool canJump(vector<int>& nums) {  
    int reach = 0;  
    for (int i = 0; i < nums.size(); i++) {  
        if (i > reach) return false;  
        reach = max(reach, i + nums[i]);  
    }  
    return true;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Jump Game II

Pattern: Greedy Range

LeetCode: #45

■ Input	non-negative jump lengths
■ Output	minimum jumps to reach the last index
■ Constraints	last index reachable
■ Main Idea / Pattern	Treat the current jump as a range. Scan the range and choose the farthest next reach.
■ Time Complexity	O(n)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
int jump(vector<int>& nums) {  
    int jumps = 0, end = 0, farthest = 0;  
    for (int i = 0; i < nums.size() - 1; i++) {  
        farthest = max(farthest, i + nums[i]);  
        if (i == end) {  
            jumps++;  
            end = farthest;  
        }  
    }  
    return jumps;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Best Time to Buy and Sell Stock II

Pattern: Greedy

LeetCode: #122

■ Input	daily stock prices
■ Output	maximum profit with unlimited transactions
■ Constraints	can buy/sell multiple times
■ Main Idea / Pattern	Take every positive day-to-day increase. Each rise contributes to the total profit.
■ Time Complexity	O(n)
■ Space Complexity	O(1)

■ Optimal C++ Code

```
int maxProfit(vector<int>& prices) {  
    int profit = 0;  
    for (int i = 1; i < prices.size(); i++)  
        if (prices[i] > prices[i - 1])  
            profit += prices[i] - prices[i - 1];  
    return profit;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Partition Labels

Pattern: Greedy + Last Occurrence

LeetCode: #763

■ Input	lowercase string
■ Output	partition lengths so each letter appears in at most one part
■ Constraints	string length ≤ 500
■ Main Idea / Pattern	Record each character's last index. Extend the current partition until all seen letters end inside it.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(1)$

■ Optimal C++ Code

```
vector<int> partitionLabels(string s) {
    vector<int> last(26);
    for (int i = 0; i < s.size(); i++)
        last[s[i] - 'a'] = i;
    vector<int> ans;
    int end = 0, start = 0;
    for (int i = 0; i < s.size(); i++) {
        end = max(end, last[s[i] - 'a']);
        if (i == end) {
            ans.push_back(end - start + 1);
            start = i + 1;
        }
    }
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Activity Selection

Pattern: Greedy + Sorting

LeetCode: ■ Absent

■ Input	activities with start/end times
■ Output	maximum number of non-overlapping activities
■ Constraints	valid intervals
■ Main Idea / Pattern	Sort by finishing time. Always choose the activity that finishes earliest among feasible choices.
■ Time Complexity	$O(n \log n)$
■ Space Complexity	$O(1)$ excluding sort

■ Optimal C++ Code

```
int maxActivities(vector<pair<int, int>>& a) {
    sort(a.begin(), a.end(),
        [](auto& x, auto& y) { return x.second < y.second; });

    int count = 0;
    int lastEnd = INT_MIN;

    for (auto& [start, end] : a) {
        if (start >= lastEnd) {
            count++;
            lastEnd = end;
        }
    }
    return count;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Jump Game

Pattern: Greedy Reach

LeetCode: 55

Input	nums
Output	whether the last index is reachable
Constraints	1 <= nums.size() <= 1e4
Main Idea / Pattern	Track the farthest reachable index; if the current index is beyond it, progress is impossible.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    bool canJump(vector<int>& nums) {
        int reach = 0;
        for (int i = 0; i < nums.size(); i++) {
            if (i > reach) return false;
            reach = max(reach, i + nums[i]);
        }
        return true;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Jump Game II

Pattern: Greedy Farthest Boundary

LeetCode: 45

Input	nums
Output	minimum jumps to reach the last index
Constraints	1 <= nums.size() <= 1e4
Main Idea / Pattern	Treat each jump as expanding a reachable range; jump when the current range ends.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int jump(vector<int>& nums) {
        int jumps = 0;
        int end = 0;
        int farthest = 0;
        for (int i = 0; i < nums.size() - 1; i++) {
            farthest = max(farthest, i + nums[i]);
            if (i == end) {
                jumps++;
                end = farthest;
            }
        }
        return jumps;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Gas Station

Pattern: Greedy Reset

LeetCode: 134

Input	gas, cost
Output	starting station index or -1
Constraints	gas.size() == cost.size(); n >= 1
Main Idea / Pattern	If total gas is enough, whenever running fuel becomes negative, the next station must be the new candidate start.
Time Complexity	O(n)
Space Complexity	O(1)

Optimal C++ Code

```
class Solution {
public:
    int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
        int total = 0;
        int tank = 0;
        int start = 0;

        for (int i = 0; i < gas.size(); i++) {
            int gain = gas[i] - cost[i];
            total += gain;
            tank += gain;

            if (tank < 0) {
                tank = 0;
                start = i + 1;
            }
        }

        return total >= 0 ? start : -1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Assign Cookies

Pattern: Sort + Greedy

LeetCode: 455

Input	g, s
Output	maximum number of children satisfied
Constraints	1 <= g.size(), s.size() <= 3e4
Main Idea / Pattern	Sort both arrays and satisfy the least greedy child with the smallest sufficient cookie.
Time Complexity	O(n log n)
Space Complexity	O(1) extra

Optimal C++ Code

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int i = 0;
        for (int cookie : s) {
            if (i < g.size() && cookie >= g[i]) {
                i++;
            }
        }
        return i;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.